

Purpose

This document contains extensive examples of SQL SELECT statements using the AdventureWorks database. These examples are designed for trainees to understand beginner, intermediate, and advanced querying concepts.

The scripts are written in ANSI SQL style and are compatible with PostgreSQL with minor modifications.

1. Basic SELECT Statements

Select All Columns

```
SELECT *  
FROM Person.Person;
```

Select Specific Columns

```
SELECT BusinessEntityID,  
FirstName,  
LastName  
FROM Person.Person;
```

Column Alias

```
SELECT FirstName AS "First Name",  
LastName AS "Last Name"  
FROM Person.Person;
```

Arithmetic Operations

```
SELECT ProductID,  
Name,  
ListPrice,  
ListPrice + 100 AS IncreasedPrice  
FROM Production.Product;
```

Concatenation

```
SELECT FirstName || ' ' || LastName AS FullName
FROM Person.Person;
```

2. DISTINCT

Unique Cities

```
SELECT DISTINCT City
FROM Person.Address;
```

Multiple Column DISTINCT

```
SELECT DISTINCT City,
PostalCode
FROM Person.Address;
```

3. WHERE Clause

Equality

```
SELECT *
FROM HumanResources.Employee
WHERE Gender = 'M';
```

Greater Than

```
SELECT *
FROM Production.Product
WHERE ListPrice > 1000;
```

Less Than

```
SELECT *
FROM Production.Product
WHERE SafetyStockLevel < 500;
```

Not Equal

```
SELECT *  
FROM Production.Product  
WHERE Color <> 'Black';
```

AND Condition

```
SELECT *  
FROM Production.Product  
WHERE Color = 'Black'  
AND ListPrice > 1000;
```

OR Condition

```
SELECT *  
FROM Production.Product  
WHERE Color = 'Black'  
OR Color = 'Silver';
```

BETWEEN

```
SELECT *  
FROM Production.Product  
WHERE ListPrice BETWEEN 500 AND 1000;
```

IN Operator

```
SELECT *  
FROM Production.Product  
WHERE Color IN ('Black', 'Red', 'Silver');
```

NOT IN

```
SELECT *  
FROM Production.Product  
WHERE Color NOT IN ('Black', 'Red');
```

LIKE

```
SELECT *  
FROM Person.Person  
WHERE FirstName LIKE 'A%';
```

LIKE with Ending Character

```
SELECT *  
FROM Person.Person  
WHERE FirstName LIKE '%n';
```

LIKE with Contains

```
SELECT *  
FROM Person.Person  
WHERE FirstName LIKE '%ar%';
```

IS NULL

```
SELECT *  
FROM Production.Product  
WHERE Color IS NULL;
```

IS NOT NULL

```
SELECT *  
FROM Production.Product  
WHERE Weight IS NOT NULL;
```

4. ORDER BY

Ascending Order

```
SELECT *  
FROM Production.Product  
ORDER BY ListPrice ASC;
```

Descending Order

```
SELECT *  
FROM Production.Product  
ORDER BY ListPrice DESC;
```

Multiple Columns

```
SELECT *  
FROM Person.Person  
ORDER BY FirstName,
```

LastName DESC;

5. LIMIT / TOP

PostgreSQL LIMIT

```
SELECT *  
FROM Production.Product  
LIMIT 10;
```

SQL Server TOP Equivalent

```
SELECT TOP 10 *  
FROM Production.Product;
```

OFFSET

```
SELECT *  
FROM Production.Product  
ORDER BY ProductID  
LIMIT 10 OFFSET 20;
```

6. Aggregate Functions

COUNT

```
SELECT COUNT(*) AS TotalProducts  
FROM Production.Product;
```

SUM

```
SELECT SUM(LineTotal) AS TotalSales  
FROM Sales.SalesOrderDetail;
```

AVG

```
SELECT AVG(ListPrice) AS AveragePrice  
FROM Production.Product;
```

MIN

```
SELECT MIN(ListPrice) AS MinimumPrice  
FROM Production.Product;
```

MAX

```
SELECT MAX(ListPrice) AS MaximumPrice  
FROM Production.Product;
```

7. GROUP BY

Total Products by Color

```
SELECT Color,  
COUNT(*) AS ProductCount  
FROM Production.Product  
GROUP BY Color;
```

Average Price by Color

```
SELECT Color,  
AVG(ListPrice) AS AvgPrice  
FROM Production.Product  
GROUP BY Color;
```

8. HAVING Clause

HAVING with COUNT

```
SELECT Color,  
COUNT(*) AS ProductCount  
FROM Production.Product  
GROUP BY Color  
HAVING COUNT(*) > 5;
```

HAVING with AVG

```
SELECT Color,
```

```
AVG(ListPrice) AS AvgPrice
FROM Production.Product
GROUP BY Color
HAVING AVG(ListPrice) > 500;
```

9. INNER JOIN

Employee with Person Details

```
SELECT e.BusinessEntityID,
p.FirstName,
p.LastName,
e.JobTitle
FROM HumanResources.Employee e
INNER JOIN Person.Person p
ON e.BusinessEntityID = p.BusinessEntityID;
```

10. LEFT JOIN

```
SELECT p.BusinessEntityID,
p.FirstName,
a.City
FROM Person.Person p
LEFT JOIN Person.BusinessEntityAddress bea
ON p.BusinessEntityID = bea.BusinessEntityID
LEFT JOIN Person.Address a
ON bea.AddressID = a.AddressID;
```

11. RIGHT JOIN

```
SELECT p.FirstName,
a.City
FROM Person.Person p
RIGHT JOIN Person.BusinessEntityAddress bea
ON p.BusinessEntityID = bea.BusinessEntityID
RIGHT JOIN Person.Address a
ON bea.AddressID = a.AddressID;
```

12. FULL OUTER JOIN

```
SELECT p.FirstName,  
a.City  
FROM Person.Person p  
FULL OUTER JOIN Person.BusinessEntityAddress bea  
ON p.BusinessEntityID = bea.BusinessEntityID  
FULL OUTER JOIN Person.Address a  
ON bea.AddressID = a.AddressID;
```

13. SELF JOIN

Employees and Their Managers

```
SELECT e1.BusinessEntityID AS EmployeeID,  
e2.BusinessEntityID AS ManagerID  
FROM HumanResources.Employee e1  
JOIN HumanResources.Employee e2  
ON e1.OrganizationNode.GetAncestor(1) = e2.OrganizationNode;
```

14. CROSS JOIN

```
SELECT p.Name,  
c.Name  
FROM Production.Product p  
CROSS JOIN Production.ProductCategory c;
```

15. Subqueries

Scalar Subquery

```
SELECT Name,  
ListPrice
```



```
FROM Production.Product
WHERE ListPrice > (
SELECT AVG(ListPrice)
FROM Production.Product
);
```

IN Subquery

```
SELECT *
FROM Sales.Customer
WHERE CustomerID IN (
SELECT CustomerID
FROM Sales.SalesOrderHeader
);
```

EXISTS

```
SELECT *
FROM Sales.Customer c
WHERE EXISTS (
SELECT 1
FROM Sales.SalesOrderHeader s
WHERE c.CustomerID = s.CustomerID
);
```

16. CASE Statement

```
SELECT Name,
ListPrice,
CASE
WHEN ListPrice > 1000 THEN 'Premium'
WHEN ListPrice BETWEEN 500 AND 1000 THEN 'Standard'
ELSE 'Budget'
END AS ProductCategory
FROM Production.Product;
```

17. UNION

```
SELECT FirstName AS Name
FROM Person.Person
```

```
UNION
SELECT Name
FROM Production.Product;
```

18. UNION ALL

```
SELECT Color
FROM Production.Product
UNION ALL
SELECT Color
FROM Production.Product;
```

19. INTERSECT

```
SELECT ProductID
FROM Sales.SalesOrderDetail
INTERSECT
SELECT ProductID
FROM Purchasing.ProductVendor;
```

20. EXCEPT

```
SELECT ProductID
FROM Production.Product
EXCEPT
SELECT ProductID
FROM Sales.SalesOrderDetail;
```

21. Common Table Expression (CTE)

```
WITH ExpensiveProducts AS (
SELECT ProductID,
Name,
ListPrice
FROM Production.Product
WHERE ListPrice > 1000
```

```
)  
SELECT *  
FROM ExpensiveProducts;
```

22. Recursive CTE

```
WITH Numbers AS (  
  SELECT 1 AS Num  
  UNION ALL  
  SELECT Num + 1  
  FROM Numbers  
  WHERE Num < 10  
)  
SELECT *  
FROM Numbers;
```

23. Window Functions

ROW_NUMBER

```
SELECT ProductID,  
       Name,  
       ListPrice,  
       ROW_NUMBER() OVER (ORDER BY ListPrice DESC) AS RowNum  
FROM Production.Product;
```

RANK

```
SELECT ProductID,  
       Name,  
       ListPrice,  
       RANK() OVER (ORDER BY ListPrice DESC) AS RankNum  
FROM Production.Product;
```

DENSE_RANK

```
SELECT ProductID,  
       Name,  
       ListPrice,
```

```
DENSE_RANK() OVER (ORDER BY ListPrice DESC) AS DenseRankNum  
FROM Production.Product;
```

NTILE

```
SELECT ProductID,  
Name,  
ListPrice,  
NTILE(4) OVER (ORDER BY ListPrice DESC) AS Quartile  
FROM Production.Product;
```

24. LEAD and LAG

LEAD

```
SELECT ProductID,  
ListPrice,  
LEAD(ListPrice) OVER (ORDER BY ProductID) AS NextPrice  
FROM Production.Product;
```

LAG

```
SELECT ProductID,  
ListPrice,  
LAG(ListPrice) OVER (ORDER BY ProductID) AS PreviousPrice  
FROM Production.Product;
```

25. FIRST_VALUE and LAST_VALUE

```
SELECT ProductID,  
ListPrice,  
FIRST_VALUE(ListPrice)  
OVER (ORDER BY ListPrice DESC) AS HighestPrice  
FROM Production.Product;
```

26. PARTITION BY

```
SELECT Color,  
Name,  
ListPrice,  
AVG(ListPrice) OVER (PARTITION BY Color) AS AvgColorPrice  
FROM Production.Product;
```

27. Correlated Subquery

```
SELECT p1.Name,  
p1.ListPrice  
FROM Production.Product p1  
WHERE p1.ListPrice > (  
SELECT AVG(p2.ListPrice)  
FROM Production.Product p2  
WHERE p1.Color = p2.Color  
);
```

28. ANY and ALL

ANY

```
SELECT Name,  
ListPrice  
FROM Production.Product  
WHERE ListPrice > ANY (  
SELECT ListPrice  
FROM Production.Product  
WHERE Color = 'Red'  
);
```

ALL

```
SELECT Name,  
ListPrice  
FROM Production.Product  
WHERE ListPrice > ALL (  
SELECT ListPrice  
FROM Production.Product  
WHERE Color = 'Red'
```

);

29. String Functions

UPPER

```
SELECT UPPER(FirstName)
FROM Person.Person;
```

LOWER

```
SELECT LOWER(FirstName)
FROM Person.Person;
```

LENGTH

```
SELECT FirstName,
LENGTH(FirstName)
FROM Person.Person;
```

SUBSTRING

```
SELECT SUBSTRING(FirstName, 1, 3)
FROM Person.Person;
```

30. Numeric Functions

ROUND

```
SELECT ROUND(ListPrice, 2)
FROM Production.Product;
```

CEILING

```
SELECT CEILING(ListPrice)
FROM Production.Product;
```

FLOOR

```
SELECT FLOOR(ListPrice)
FROM Production.Product;
```

31. Date Functions

Current Date

```
SELECT CURRENT_DATE;
```

Extract Year

```
SELECT EXTRACT(YEAR FROM OrderDate)
FROM Sales.SalesOrderHeader;
```

Age

```
SELECT AGE(CURRENT_DATE, HireDate)
FROM HumanResources.Employee;
```

32. COALESCE

```
SELECT Name,
COALESCE(Color, 'No Color') AS ProductColor
FROM Production.Product;
```

33. NULLIF

```
SELECT NULLIF(ListPrice, 0)
FROM Production.Product;
```

34. CAST and CONVERT

CAST

```
SELECT CAST(ListPrice AS INTEGER)
FROM Production.Product;
```

PostgreSQL Type Conversion

```
SELECT ListPrice::INTEGER
FROM Production.Product;
```

35. Pagination

```
SELECT *
FROM Sales.SalesOrderHeader
ORDER BY SalesOrderID
LIMIT 20 OFFSET 40;
```

36. Advanced Reporting Query

Top 10 Highest Revenue Products

```
SELECT p.Name,
SUM(sod.LineTotal) AS Revenue
FROM Sales.SalesOrderDetail sod
INNER JOIN Production.Product p
ON sod.ProductID = p.ProductID
GROUP BY p.Name
ORDER BY Revenue DESC
```

I've prepared a complete AdventureWorks-based SELECT statement master script for training purposes. It covers:

- Basic SELECT
- WHERE conditions
- ORDER BY
- DISTINCT
- Aggregations
- GROUP BY / HAVING
- All JOIN types
- Subqueries
- CTEs
- Window functions

- CASE statements
- UNION / INTERSECT / EXCEPT
- String, numeric, and date functions
- Pagination
- Real-time reporting queries
- Performance tips
- Execution order of SELECT
- Practice exercises
- Interview questions

The examples are structured progressively from beginner to advanced level using AdventureWorks tables and realistic business scenarios.